

Passly

Informe de la Obligatoria 1

Desarrollo de Aplicaciones II - Comision Lunes TM

Fecha de entrega: 14/09/2026

Equipo: Federico Torcini, Simon Forteza, Juan Segundo Addamo y Lucio Dillon

Version de trabajo: corte verificable del 12/09/2026 ([main b639aa9](#))

Este documento diferencia lo implementado de lo planificado y declara de forma explicita los requisitos que aun permanecen pendientes.

Resumen ejecutivo

Passly es una plataforma para vender y validar entradas de eventos. El backend se diseña como un monolito modular: una sola aplicacion desplegable, dividida en componentes de negocio con contratos explicitos y fronteras verificadas mediante Spring Modulith.

Al corte de este informe estan implementados los componentes Usuarios, Productoras y Eventos, todos con capas de presentacion, negocio y datos, persistencia en esquemas PostgreSQL separados y contratos que evitan exponer entidades JPA. PAS-6 agrega autenticacion HTTP Basic stateless y autorizacion declarativa por rol. Ventas como componente stateful y Facade, y Pagos como Adapter REST, permanecen planificados; por lo tanto, esos requisitos no se presentan como cumplidos.

1. Problema, alcance y arquitectura

Passly cubre el recorrido que comienza cuando un organizador publica un evento y termina cuando un validador acepta un ticket en la puerta. Un comprador selecciona entradas, realiza un pago, recibe un ticket con QR y posteriormente ese QR se marca como utilizado para impedir el doble ingreso.

La Obligatoria 1 se concentra en demostrar decisiones de arquitectura defendibles y funcionalidad incremental. La cantidad de código no reemplaza la necesidad de justificar componentes, estados, patrones, seguridad y fronteras.

1.1 Monolito modular

El backend se ejecuta como un único artefacto Spring Boot, pero no se organiza como un monolito sin límites. Cada componente publica una interfaz y mantiene sus controladores, implementaciones, entidades y repositorios bajo `internal`.

Spring Modulith analiza las dependencias y hace fallar el build ante accesos indebidos o ciclos. Esta decisión conserva transacciones locales y evita introducir sagas, fallos de red y despliegues coordinados antes de que exista una necesidad real.

La decisión completa se registra en ADR-001: Monolito modular en lugar de microservicios.

1.2 Persistencia por componente

Se utiliza una única instancia PostgreSQL con un esquema por componente. Eventos, Productoras y Usuarios poseen esquemas separados. Un componente no puede consultar directamente las tablas de otro: debe utilizar su interfaz pública. Eventos conserva `productora_id` y Productoras conserva `usuario_id` como referencias lógicas, sin claves foráneas ni joins entre esquemas.

La decisión reduce complejidad operativa y mantiene visible la propiedad de los datos. El detalle se registra en ADR-002: Esquema PostgreSQL por componente.

1.3 Tecnologías principales

- Java 21 y Spring Boot 4.1.1.
- Spring Modulith 2.1.1.
- Spring Data JPA e Hibernate.
- PostgreSQL local mediante Docker Compose, sustituible por Supabase mediante configuración.
- Maven para build y pruebas.

2. Componentes y contratos

2.1 ServicioDeEventos - implementado

Eventos administra el alta y publicacion de eventos, tipos de entrada, precios, cupos y disponibilidad. Cada evento pertenece a una productora. Expone EventoService y DTO publicos; la implementacion de negocio, el controlador y la persistencia quedan encapsulados.

La demostracion permite crear y publicar eventos, consultar la cartelera completa o filtrada y verificar aislamiento: un ORGANIZADOR solo opera sobre eventos de una productora que puede gestionar. Eventos obtiene la identidad autenticada desde Spring Security y delega en ProductoraService la comprobacion de membresia.

2.2 ServicioDeUsuarios - implementado

Usuarios administra el registro y la consulta de identidades con los roles COMPRADOR, ORGANIZADOR, VALIDADOR y ADMIN. Expone UsuarioService, mientras que la entidad, el repositorio, el mapper y el encoder permanecen internos.

Las credenciales se transforman con BCrypt antes de persistirse. El hash no forma parte de UsuarioDTO y nunca se devuelve por la API. Seguridad consume el contrato usuarios :: autenticacion para buscar credenciales sin acceder al repositorio interno.

2.3 ServicioDeProductoras - implementado

Productoras administra las organizaciones y su padron. Distingue el rol global del usuario del rol interno DUENIO, STAFF o VALIDADOR, valida su compatibilidad y expone ProductoraService. El grafo de negocio verificado queda eventos -> productoras -> usuarios.

La cartelera contiene eventos de distintas productoras y el backoffice rechaza con 403 a miembros de otra organizacion o a miembros sin capacidad de gestion. Como deuda tecnica declarada, los endpoints propios de Productoras todavia reciben el header temporal X-Usuario-Id; Eventos ya fue migrado a la identidad autenticada.

2.4 Seguridad - implementada

El modulo Seguridad configura HTTP Basic, sesion STATELESS, CSRF deshabilitado y reglas por rol con @PreAuthorize. Reutiliza el PasswordEncoder de Usuarios y representa el id numerico dentro del UserDetails, de modo que los controladores puedan identificar al actor sin introducir dependencias de negocio indebidas.

2.5 Componentes previstos

- **ServicioDeVentas:** orquesta la compra y mantiene temporalmente la seleccion del comprador. Debe implementar el estado conversacional y actuar como Facade.
- **ServicioDePagos:** oculta el contrato de una pasarela externa detras de una interfaz propia. Debe implementar el patron Adapter.

2.6 Componentes posteriores

Tickets emitira entradas con QR; Accesos validara su uso unico; Notificaciones enviara mensajes; Facturacion adaptara una integracion SOAP. Estos elementos pertenecen principalmente a entregas posteriores y no se presentan como implementados en este corte.

3. Capas, estado y patrones

3.1 Arquitectura en tres capas

Cada componente sigue el flujo presentacion -> negocio -> datos.

- **Presentacion:** traduce HTTP a solicitudes de negocio, valida la forma de entrada y transforma excepciones en respuestas HTTP.
- **Negocio:** contiene reglas, orquestacion y limites transaccionales.
- **Datos:** implementa persistencia mediante entidades JPA y repositorios Spring Data.

Las entidades no cruzan la frontera del componente. Los intercambios se realizan mediante contratos y DTO.

3.2 Stateful y stateless

Eventos, Productoras, Usuarios y Seguridad son stateless: no conservan estado conversacional entre llamadas.

Persistir informacion de dominio no vuelve stateful al componente, porque ese estado vive en PostgreSQL.

Seguridad tampoco crea sesion HTTP; cada request vuelve a presentar sus credenciales.

Ventas se diseña como stateful porque debe mantener un hold temporal mientras el comprador completa el pago.

Ese estado en memoria crece durante la conversacion y expira si se abandona. La implementacion y sus callbacks de ciclo de vida permanecen pendientes; hoy no existe un componente stateful integrado.

3.3 Patrones de diseño

DAO - implementado

Los repositorios Spring Data separan la logica de negocio del acceso a PostgreSQL. Los servicios dependen de repositorios y no escriben SQL ni administran conexiones.

Facade - planificado en Ventas

VentaService ofrecera una operacion de compra simple y ocultara la coordinacion entre disponibilidad, identidad, pago y emision. Solo debe declararse implementado cuando PAS-8 este integrado y probado.

Adapter - planificado en Pagos

PagoService presentara un contrato estable del dominio y traducira solicitudes y respuestas de la pasarela REST. No existe una implementacion integrada al corte.

Los servicios de aplicacion y mappers aportan separacion estructural, pero no se contabilizan aqui para ocultar la brecha: de los tres patrones exigidos para la entrega, DAO esta aplicado y Facade/Adapter continuan pendientes.

4. Seguridad, transacciones e integraciones

4.1 Autenticacion stateless

Usuarios valida el registro, rechaza emails repetidos y persiste hashes BCrypt.

DetalleDeUsuarioParaAutenticacion implementa `UserDetailsService`, consulta el contrato publico de Usuarios y entrega las credenciales a un `DaoAuthenticationProvider`. El login se realiza por email, pero el principal autenticado conserva el id numerico del usuario para las reglas de alcance.

4.2 Autorizacion declarativa implementada

El `SecurityFilterChain` permite la cartelera, el healthcheck y el alta publica de compradores; el resto exige autenticacion. `@PreAuthorize` protege operaciones sensibles:

- Solo ORGANIZADOR puede crear o publicar eventos, y el servicio exige ademas que gestione la productora correspondiente.
- Un alta anonima solo puede registrar COMPRADOR; los roles privilegiados requieren ADMIN.
- Solo ADMIN puede listar usuarios.

La evidencia reproducible esta en `docs/http/07-seguridad.http`: principal anonimo rechazado con 401, usuario autenticado con rol insuficiente rechazado con 403 y accesos autorizados con 200/201. `AutorizacionPorRolTest` fija el caso de un COMPRADOR autenticado que no puede crear eventos.

4.3 Transaccion critica

La confirmacion futura de una compra coordinara descuento de cupo, registro de pago y emision de tickets. Esa secuencia debe ser atomica: si falla un paso, se revierte el conjunto.

La llamada externa de facturacion queda fuera de la transaccion. Mantener una conexion a AFIP dentro de `@Transactional` bloquearia recursos y no permitiria revertir el sistema externo. La venta se confirma localmente y la factura se procesara luego mediante mensajeria con reintentos.

4.4 Integraciones previstas

- REST saliente hacia la pasarela de pago.
- Cola punto a punto `orden.pagada` para Facturacion.
- Topico `ticket.emitido` para Notificaciones y Accesos.
- SOAP con un servicio AFIP simulado.

5. Verificacion y demostracion

5.1 Verificacion automatica

EstructuraDeModulosTest ejecuta cuatro verificaciones: ausencia de violaciones, Usuarios como raiz, grafo eventos -> productoras -> usuarios y generacion de documentacion. El 12/09 se ejecuto en Java 21: 4 pruebas, 0 fallos y build exitoso, con Usuarios, Productoras, Eventos y Seguridad detectados como modulos.

La suite completa compila, pero en esta verificacion finalizo con 2 errores de contexto porque no habia PostgreSQL escuchando en localhost:5432; no fueron fallos de asercion. Debe repetirse con la infraestructura levantada. Actualmente el repositorio no posee un workflow de integracion continua.

5.2 Guion de demostracion

- 1 Levantar PostgreSQL con `docker compose up -d`.
- 2 Iniciar el backend con el perfil demo.
- 3 Consultar `/actuator/health` y `/actuator/modulith`.
- 4 Ejecutar `07-seguridad.http` para demostrar 401, 403 y accesos permitidos por rol.
- 5 Ejecutar `06-aislamiento.http` para demostrar que una productora no gestiona eventos de otra.
- 6 Consultar la cartelera completa y filtrada por productora.
- 7 Registrar y consultar un usuario, verificando que el hash no aparece en la respuesta.
- 8 Ejecutar `mvnw test` con PostgreSQL disponible y conservar el resultado como evidencia.

5.3 Pendientes para cerrar la entrega

- Implementar Ventas/PAS-8 como componente stateful y Facade.
- Implementar el Adapter de Pagos si se lo incluye en el alcance de la Obligatoria 1.
- Migrar Productoras desde X-Usuario-Id hacia el principal autenticado.
- Ejecutar la suite completa con PostgreSQL y conservar el resultado exitoso.
- Ejecutar la demo completa en la maquina que se utilizara en la defensa.
- Realizar una revision cruzada: cada integrante debe explicar cualquier componente.

6. Uso de inteligencia artificial y conclusiones

6.1 Declaracion de uso de IA

El equipo declara que utilizo herramientas de inteligencia artificial durante el desarrollo de Passly. Su uso se informa con fines de transparencia academica y no sustituye el trabajo, la revision ni la responsabilidad de los integrantes.

Herramientas utilizadas

- **Claude (Anthropic):** asistencia para analizar la consigna, estructurar el roadmap, dividir el trabajo en issues y apoyar la implementacion y documentacion tecnica.
- **Codex (OpenAI):** inspeccion del repositorio y su historial, contraste del estado de implementacion, redaccion de los ADR y del informe, generacion reproducible del PDF y controles de consistencia y presentacion.

Alcance del uso

Las herramientas intervinieron en las etapas de interpretacion de requisitos, evaluacion de alternativas de arquitectura, planificacion, apoyo a la escritura y revision de codigo, preparacion de pruebas y elaboracion de documentacion. Las decisiones finales de arquitectura, el codigo aceptado y la seleccion del contenido presentado fueron tomadas por el equipo.

Procesos de validacion

Los resultados generados o sugeridos fueron revisados por integrantes del equipo, contrastados con la consigna y con documentacion oficial, y corregidos cuando no coincidian con el estado real del proyecto. El codigo se valida mediante compilacion, pruebas de fronteras con Spring Modulith, pruebas de API y demostraciones contra PostgreSQL. La documentacion se compara con el codigo integrado en `main` y se revisa mediante pull requests. Ningun resultado se considera valido solo por haber sido producido por una IA.

Responsabilidad sobre el contenido

Los integrantes asumen plena responsabilidad academica por el codigo, las decisiones y la documentacion presentados. Cada miembro debe comprenderlos, poder explicar sus fundamentos y defender cualquier parte durante la evaluacion oral, aunque haya recibido asistencia de IA.

La declaracion no implica por si misma una penalizacion. La omision del uso de estas herramientas o la presentacion de informacion falsa o engañosa puede considerarse una falta a la integridad academica. Antes de entregar, cada integrante debe confirmar que esta lista incluye todas las herramientas que realmente utilizo y que el alcance informado es correcto.

6.2 Conclusiones

La arquitectura elegida busca equilibrio entre separacion y simplicidad. El monolito modular permite contratos claros, transacciones locales y una demostracion reproducible sin asumir prematuramente los costos de un sistema distribuido. El esquema PostgreSQL por componente refleja la misma decision en la persistencia.

Usuarios, Productoras y Eventos demuestran tres rebanadas verticales y Seguridad cubre autenticacion y autorizacion por rol. Spring Modulith verifica fronteras y el grafo de dependencias. Para cumplir integralmente la vara de la Obligatoria 1 aun deben incorporarse un componente stateful y los patrones Facade y Adapter; ademas debe repetirse la suite completa con PostgreSQL y ensayarse la defensa.

Referencias

- Repositorio Passly: <https://github.com/SimonForteza/Passly>
- Spring Boot: <https://docs.spring.io/spring-boot/index.html>
- Spring Modulith: <https://docs.spring.io/spring-modulith/reference/index.html>
- PostgreSQL: <https://www.postgresql.org/docs/>
- Documentacion interna: `CLAUDE.md`